DAY 29 / 90

AWS Week — Day 7: Highly Available App

Capstone • ALB • Auto Scaling • Multi-AZ • RDS • CloudWatch

Goal: Combine this week's AWS skills to design and, if budget allows, deploy a web application across two Availability Zones behind an Application Load Balancer. Connect it to a private database, monitor it, and prove it recovers when one app instance fails.

Source mapping: Week 3 AWS source document — Day 7 Capstone: Highly Available App, Challenge Day 29/90

Scope rule: Do not repeat Days 22–28 individually. Reuse the Day 27 VPC, Day 25 EC2 bootstrap and Day 26 private RDS only if they still exist and are safe to reuse. Focus on integrating them into a highly available system.

Cost warning: ALB, NAT Gateway, RDS, EC2, public IPv4, CloudWatch and data transfer may cost money. Check current pricing. Do not run overnight. If unsure, complete the design-only path. A budget alert is not a hard spending cap.

1. Target architecture

```
Users -> ALB (public subnets, AZ-A + AZ-B)
| health check
+--------+--------+
v v
EC2 app A EC2 app B
(private AZ-A) (private AZ-B)
\ /
+----> Private RDS PostgreSQL

Auto Scaling Group: min 2, desired 2, max 4
CloudWatch: CPU / 5xx / UnHealthyHostCount
Security groups: alb-sg -> app-sg -> db-sg
```

ALB receives user traffic. Health checks identify healthy app instances. Auto Scaling maintains capacity across two AZs. The database remains private.

2. Key concepts

Component	Purpose
ALB	Layer 7 HTTP/HTTPS load balancer; distributes traffic to healthy targets.
Target group	Defines backend targets and health-check settings.
Launch template	Stores AMI, instance type, security group, instance profile and user data.
Auto Scaling group	Maintains desired EC2 capacity, replaces unhealthy instances and scales on metrics.
Multi-AZ	Spreads app capacity across AZs to reduce a single-AZ failure risk.
CloudWatch	Collects metrics/logs and evaluates alarms; can notify through SNS or support scaling.

3. Task A — Pre-flight planning

• Confirm the Day 27 VPC has public and private subnets across two AZs. Reuse it only if it is still available and correctly configured.

• Confirm the Day 26 RDS is private and available. If deleted, use the design-only path rather than recreating it without checking costs.

• Plan alb-sg, app-sg and db-sg. Traffic chain: internet → ALB → app → database.

• Prepare the Day 25 Nginx bootstrap script so the page displays the EC2 instance ID.

• Set a same-day test and cleanup window; record region, resource names and cost assumptions.

4. Task B — Security group chain

Security group	Inbound rule
alb-sg	HTTP TCP 80 from internet for this lab. Production should use HTTPS and appropriate restrictions.
app-sg	HTTP TCP 80 with source = alb-sg only.
db-sg	PostgreSQL TCP 5432 with source = app-sg only.

App instances belong in private subnets without public IPs. ALB belongs in public subnets in at least two AZs. RDS stays private.

5. Task C — Launch template

Create a launch template using a suitable Ubuntu AMI (or Day 25 AMI), the source example's t3.micro, app-sg, an SSM-capable instance profile and IMDSv2. Reuse the Day 25 bootstrap script. Encode user data as required by your chosen tool.

```
ImageId: <UBUNTU_AMI_ID>
InstanceType: t3.micro
SecurityGroupIds: [<APP_SG_ID>]
```

```
IamInstanceProfile: <SSM_INSTANCE_PROFILE>
UserData: <BASE64_USER_DATA>
MetadataOptions.HttpTokens: required
```

Replace placeholders with real lab values. Never put database passwords or access keys in user data.

6. Task D — ALB, target group and Auto Scaling

• Create an HTTP target group on port 80. Choose a health-check path actually served by your Nginx app (the source example uses /).

• Create an internet-facing ALB in both public subnets and attach alb-sg.

• Create an HTTP listener on port 80 that forwards to the target group.

• Create an ASG using the launch template in both private subnets; attach the target group and set health-check type to ELB.

• Set minimum 2, desired 2, maximum 4; allow enough health-check grace period for user data to finish.

• Wait until both instances register as healthy targets.

```
aws elbv2 describe-target-health --target-group-arn <TARGET_GROUP_ARN>
aws autoscaling describe-auto-scaling-groups --auto-scaling-group-names <ASG_NAME>
```

7. Task E — Failure and recovery test

• Copy the ALB DNS name and send repeated requests. The web page should display the instance ID.

• Confirm at least two different instance IDs appear across repeated requests.

• Terminate one instance that belongs to this lab ASG only.

• Continue sending requests. The ALB should keep serving while the ASG launches a replacement.

• Inspect ASG scaling activities and target health; record the failure, replacement and recovery evidence.

```
for i in 1 2 3 4 5 6; do
curl -s http://<ALB_DNS_NAME>
done
```

Do not terminate instances belonging to another project or shared environment.

8. Task F — Scaling and monitoring

• Create a target-tracking scaling policy based on average ASG CPU utilization; the source example uses 50%.

• If testing load, use a controlled test on your own lab instances and stop it after observing scaling.

• Optionally create an SNS topic and email subscription; confirm the email subscription.

• Create a CloudWatch alarm for 5xx errors or UnHealthyHostCount and a dashboard for requests, healthy hosts and CPU.

• Verify alarm configuration and dashboard metrics; do not claim the alarm fired unless you observed the expected state.

CloudWatch metrics and alarms may have costs depending on usage. Keep monitoring minimal.

9. Troubleshooting

Symptom	Checks
ALB 503	No healthy targets: inspect target health, health-check path, app-sg rule from alb-sg, and whether user data completed.
ALB 502	Check Nginx status, app response and instance/application logs.
Instances repeatedly replaced	Check ASG health-check type, grace period, target health and bootstrap completion.
Private instances cannot install Nginx	Private subnet needs NAT or suitable endpoints for outbound access; inspect routes and outbound rules.
ALB unreachable	Check public subnets in two AZs, ALB SG inbound HTTP, listener and target group.
RDS connection fails	Check db-sg allows 5432 from app-sg, endpoint/credentials and VPC connectivity.

10. Cleanup — mandatory

• Set ASG desired capacity to 0 or delete the ASG; confirm its instances terminate.

• Delete ALB listener, ALB and target group after the ASG no longer depends on them.

• Delete the lab launch template, CloudWatch alarms/dashboard and SNS resources.

• If you created a lab RDS, delete it and manual snapshots after confirming they are no longer needed.

• Delete lab NAT Gateways and release Elastic IPs. Remove only lab-created networking resources after checking dependencies.

• Verify no lab EC2, ALB, target group, ASG, RDS, NAT Gateway, Elastic IP or snapshots remain. Do not delete shared resources from earlier days.

Important: Deleting app instances alone does not remove the ALB, ASG, NAT Gateway, RDS or snapshots. Check each service after teardown.

11. Deliverable

• Create **day7-ha-app/** with scripts/configuration notes (never commit credentials).

• Add an architecture diagram showing two AZs and the alb-sg → app-sg → db-sg chain.

• Include target health, repeated ALB responses showing instance IDs and ASG replacement activity.

• Include CloudWatch alarm/dashboard screenshots, cost note and lessons learned.

Only 4–6 screenshots: healthy targets; repeated ALB requests with different instance IDs; ASG replacement activity; CloudWatch alarm; dashboard; cleanup proof. Hide account IDs and sensitive values.

12. Interview questions

Q. How do you design a highly available AWS web app?
Multiple AZs, ALB health checks, ASG across AZs, stateless app tier, Multi-AZ database, automated recovery and monitoring.

Q. EC2 health checks vs ELB health checks in ASG?
EC2 checks detect instance-level problems. ELB checks can detect application/target health and replace an instance whose app is unhealthy.

Q. ALB vs NLB?
ALB is Layer 7 for HTTP/HTTPS routing. NLB is Layer 4 for TCP/UDP/TLS and high-performance network traffic.

Q. ALB returns 502/503. What do you check?
Target health, health-check path, security-group rules, app logs, Nginx status and recent deployments.

Q. How does Auto Scaling decide to scale?
Policies react to CloudWatch metrics, such as target tracking, step or scheduled scaling, within min/max capacity.

Q. How do you control AWS costs?
Right-size, scale down when idle, use endpoints where appropriate, set budgets/alarms, tag resources and delete unused infrastructure.

Q. What would you add for production?
HTTPS with ACM, WAF, Route 53, centralized logs, backups, IaC and CI/CD.

13. Completion checklist

• ALB serves the app through healthy targets across two AZs.

• Repeated requests show at least two different instance IDs.

• Terminating one ASG instance causes no downtime and a replacement launches.

• CPU target tracking creates a scaling activity (if load test completed).

• CloudWatch alarm and dashboard are configured and verified.

• All lab resources are deleted and cost notes recorded.

Week 3 finish line: IAM • S3/CloudFront • EC2 • RDS • VPC • EKS • High Availability